African Scientific Research and Innovation Council

BAAI 智源研究院

北京大学 PEKING UNIVERSITY

中国人民大学 RENMIN UNIVERSITY OF CHINA

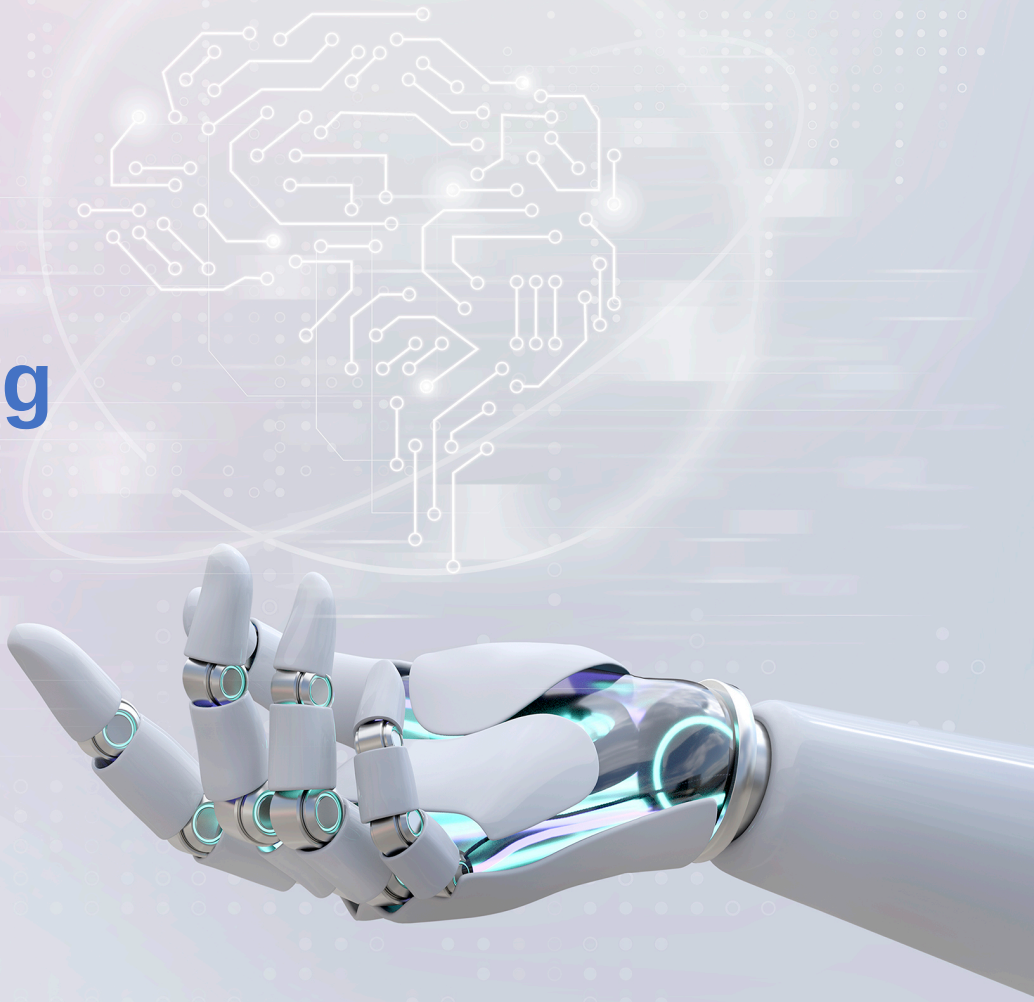
Global AI Compute Faculty Development Initiative

Module 01: AI Systems Fundamentals and Heterogeneous Computing

Guojie Luo
罗国杰
gluo@pku.edu.cn

Jing Wang
王晶
jwang@ruc.edu.cn

October 2026 · 2nd AICFD



Supported by UNDP.

Outline

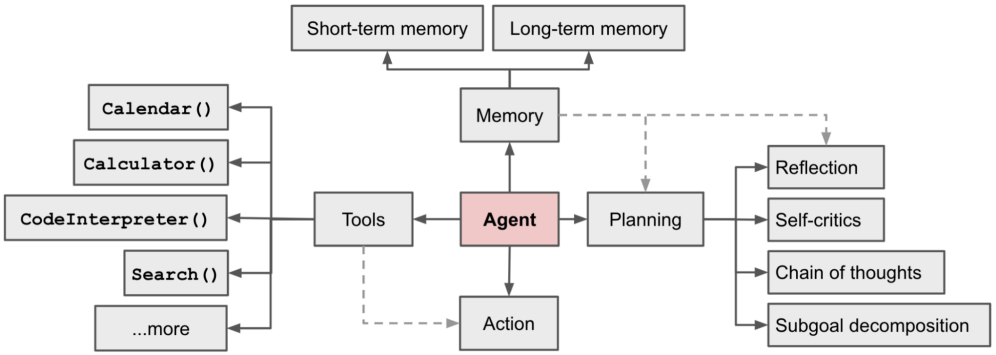
1. AI Computing and Heterogeneous Hardware
2. Heterogeneous Programming Fundamentals
3. Triton Programming Basics
4. Lab: Softmax, GEMM, and Tuning

1. AI Computing and Heterogeneous Hardware

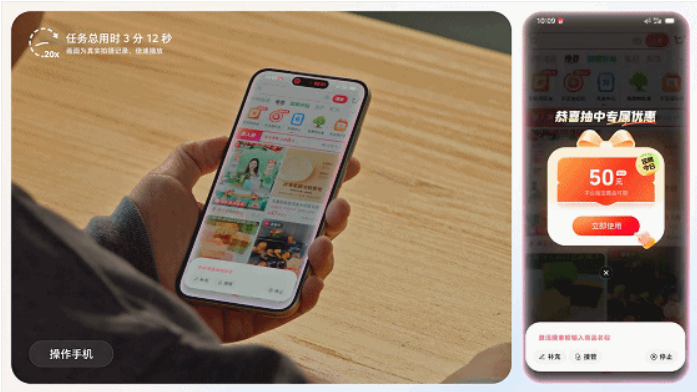
- Token-Hungry Generative AI Applications
- The Hardware Demands of AI/ML
- Limits of General-Purpose CPUs for AI
- The Rise of the GPU
- AI Hardware Spectrum & Evolving Architectures

1.1. Token-Hungry Generative AI Applications

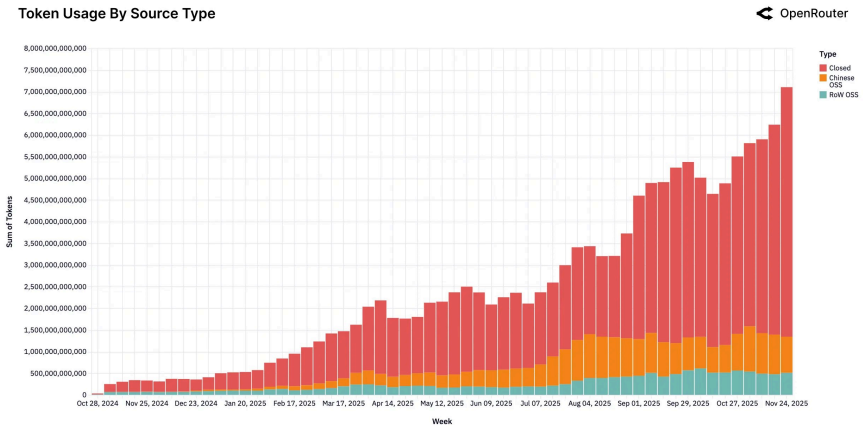
- LLM-powered autonomous agents



- AI mobile phone and AI PC



- Token consumption trends

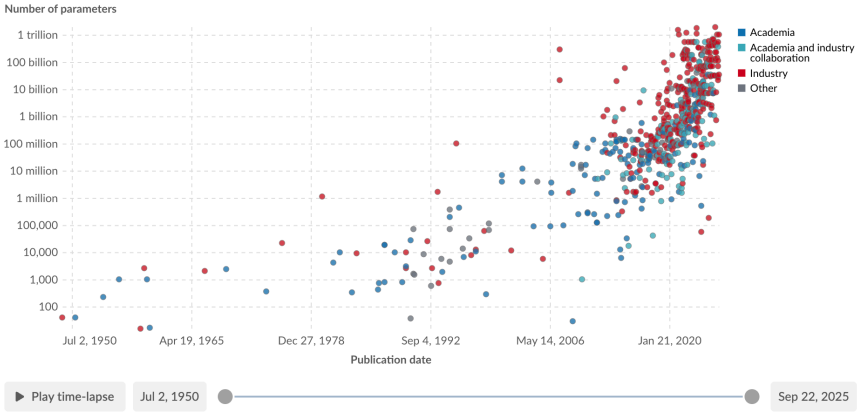


- Video generation



1.2. The Hardware Demands of AI/ML

- Model Parameters
  - The exponential growth.
- Computational Demand
  - Outpacing the Moore's Law.
  - Comparison between compute cost and U.S GDP by OpenAI.
- Communication Scale
  - The new critical bottleneck.



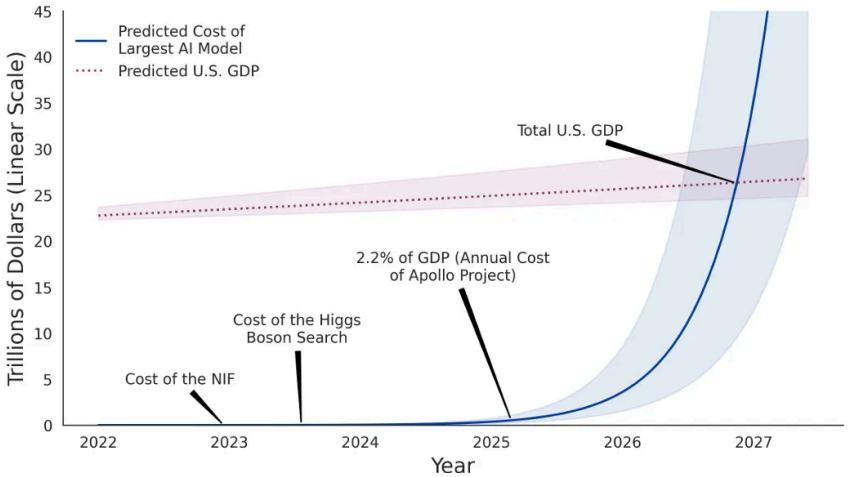
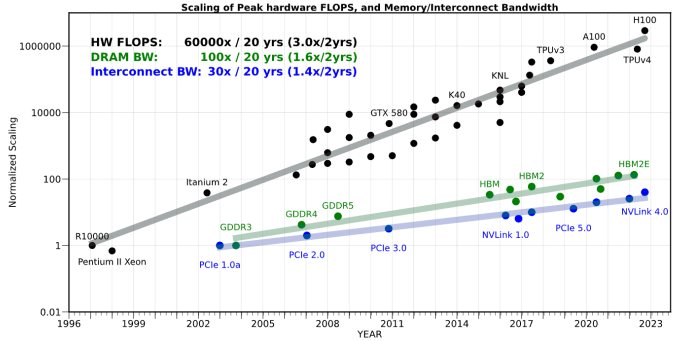
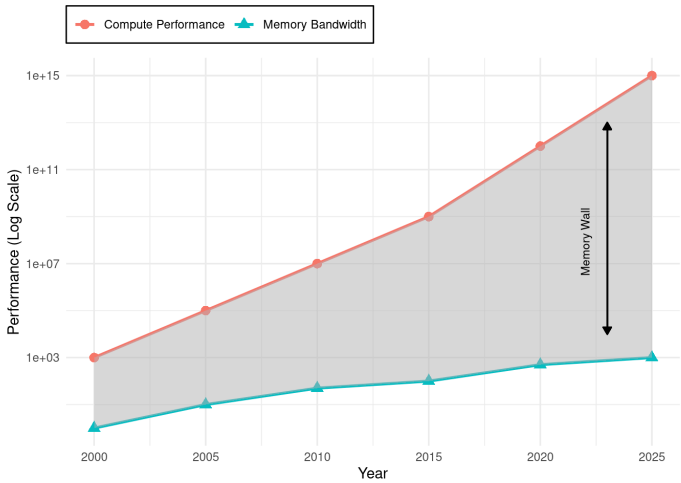


image: https://medium.com/geekculture/business-analysis-ai-computational-cost-67a136957c95
https://ourworldindata.org/grapher/artificial-intelligence-parameter-count

1.3a. Limits of General-Purpose CPUs for AI

- The CPU's Legacy:
  - For decades, the Central Processing Unit (CPU) was the universal engine of computing, designed for sequential, general-purpose tasks.
- The Two Crises:
  - **Von Neumann Bottleneck**
  - **End of Performance Scaling**

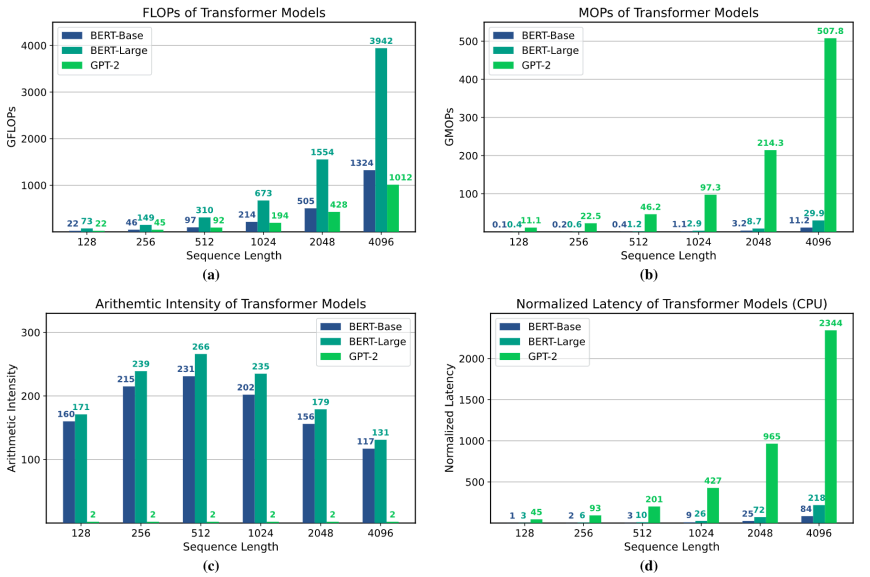


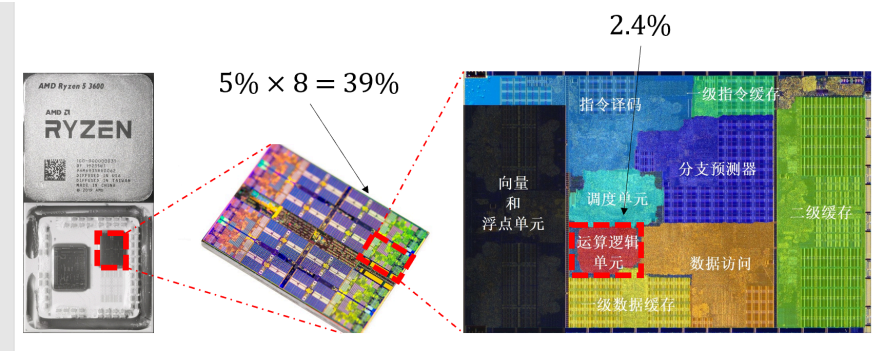


A. Gholami, Z. Yao, S. Kim, C. Hooper, M. W. Mahoney and K. Keutzer, "AI and Memory Wall," in IEEE Micro, vol. 44, no. 3, pp. 33-39, May-June 2024, doi: 10.1109/MM.2024.3373763.

1.3b. Limits of General-Purpose CPUs for AI

- The Mismatch:
  - General-purpose CPUs spend most of the die on control, caches, and ILP machinery — a poor fit for the dense, regular parallelism in AI kernels such as matrix multiplication.
  - *CPUs achieve only 5-10% utilization on neural network workloads, delivering approximately 100 GFLOPS while consuming hundreds of watts.*
- Conclusion:
  - AI's growth exposed these limits and motivated specialized accelerators **alongside** CPUs, not a claim that CPUs are obsolete.

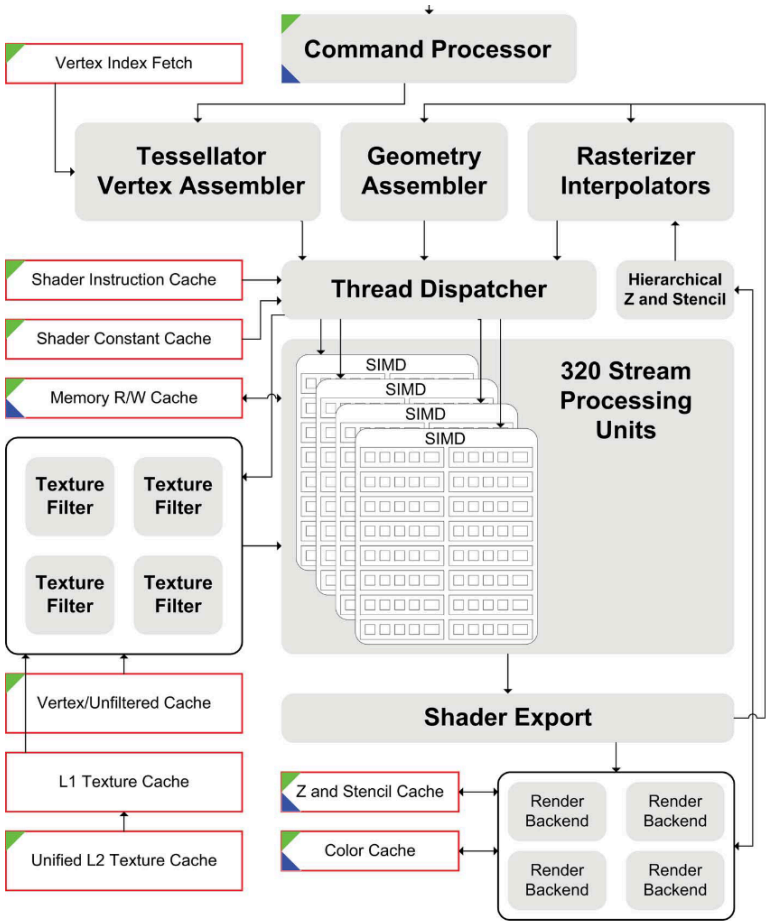




A. Gholami et al., "AI and Memory Wall," in IEEE Micro, vol. 44, no. 3, pp. 33-39, May-June 2024.

1.4a. The Rise of the GPU

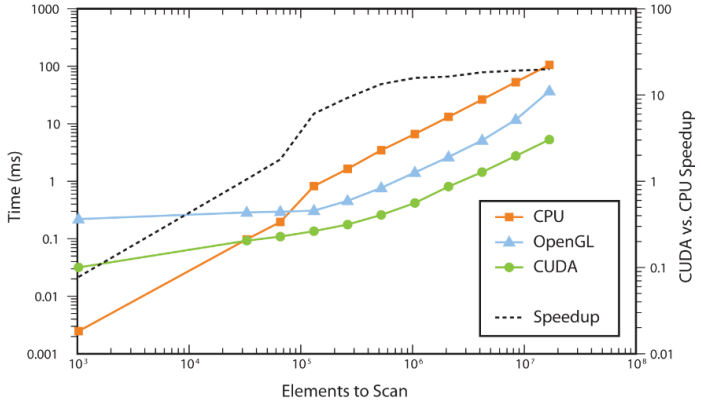
- An Unlikely Hero: The Graphics Processing Unit (GPU) was born in the late 1990s for video game graphics, featuring thousands of simple cores for parallel tasks.
- Architectural Resonance: The core math of graphics (matrix and vector operations) was identical to the math of deep learning, making GPUs a perfect, albeit accidental, fit.

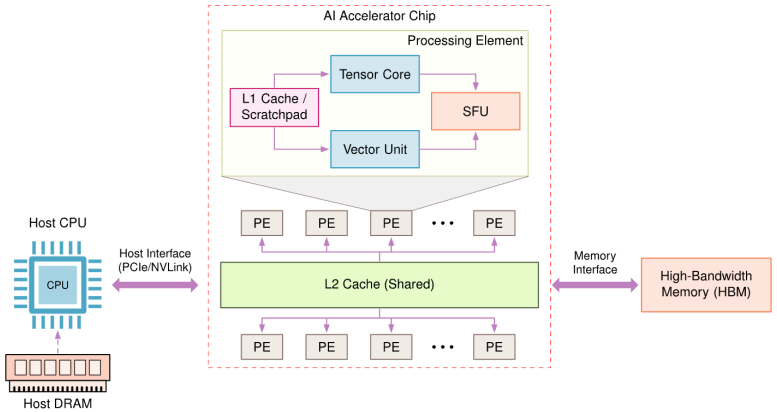


J. D. Owens, M. Houston, D. Luebke, S. Green, J. E. Stone and J. C. Phillips, “GPU Computing,” in Proceedings of the IEEE, vol. 96, no. 5, pp. 879-899, May 2008, doi: 10.1109/JPROC.2008.917757.

1.4b. The Rise of the GPU

- The Software Catalyst: CUDA: NVIDIA's CUDA platform (2007) unlocked the GPU for general-purpose computing, making its power accessible to AI researchers without deep graphics expertise.
- The Result: Training times for AI models plummeted from weeks to hours, establishing the GPU as the de facto standard for AI training.





J. D. Owens, M. Houston, D. Luebke, S. Green, J. E. Stone and J. C. Phillips, "GPU Computing," in Proceedings of the IEEE, vol. 96, no. 5, pp. 879-899, May 2008, doi: 10.1109/JPROC.2008.917757.

1.5. AI Hardware Spectrum: Key Examples

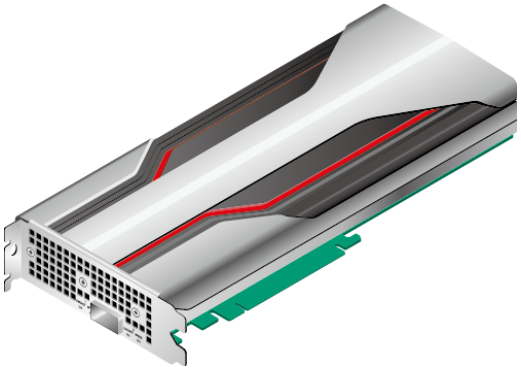
Cloud ML

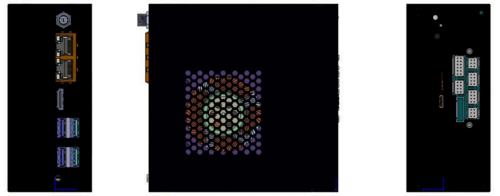
Huawei Ascend

Cambricon

MLU370

etc.





Edge ML

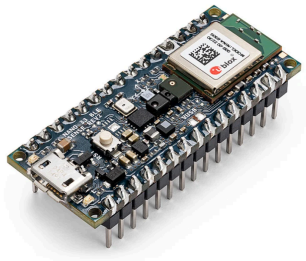
Iluvatar

TY1200

Mobile ML

Kirin 9030 Pro





Tiny ML

Arduino Nano 33

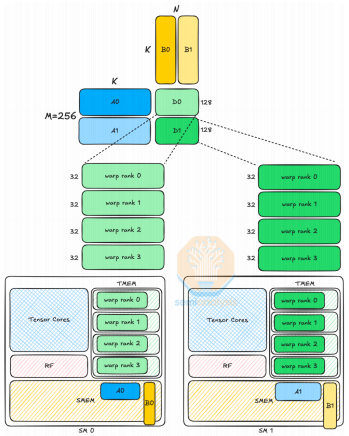
BLE Sense

1.6. Compute Architectures Evolve

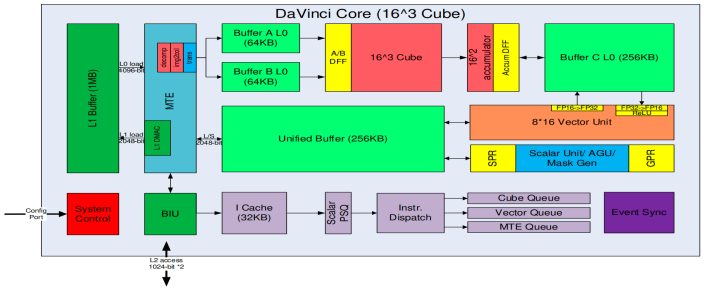
- GPU series (+ Tensor cores)
- TPU series
- Ascend series (DaVinci Cube)
- CPU series: Intel AMX; AMD (+ NPU / AIE)
- Graph / NoC NPUs (e.g. Tenstorrent)
- compute-in-memory · wafer-scale · rack-scale nodes
- …

Same math, different stacks

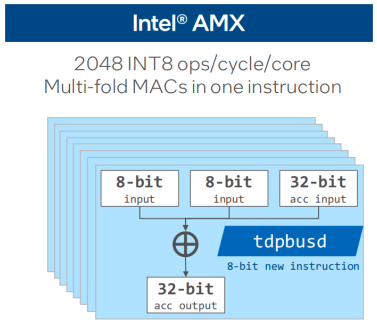
Dense GEMM/conv primitives recur; silicon differs in tiling, on-chip memory, and the programming model (CUDA / CANN / AMX / …).



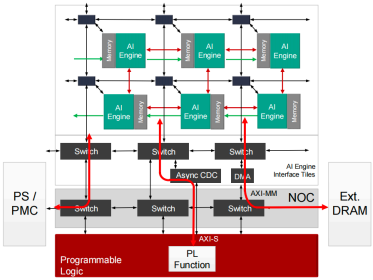
NVIDIA Tensor Core



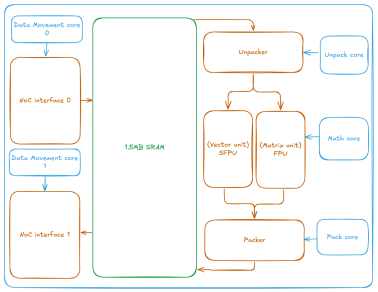
Huawei DaVinci



Intel AMX



AMD AIE



Tenstorrent

1.7. Different Processors Optimize Different Trade-offs

CPU	GPU	NPU
Optimized for low latency and complex control	Optimized for highly parallel throughput	Optimized for common AI dataflows
A small number of powerful cores	Many parallel execution lanes	Matrix and vector engines with on-chip buffers
Large caches and branch prediction	SIMT execution and high-bandwidth memory	High efficiency for supported operators and data types
Best suited for orchestration, preprocessing and control-heavy workloads	Best suited for tensor operations and programmable kernels	Usually driven by compiler, graph and framework toolchains

Modern AI systems combine CPUs and accelerators rather than relying on a single processor type.

1.8a. Iluvatar BI-V150: A General-Purpose GPU for AI Computing

- 1. Product Positioning
  - General-Purpose GPU
  - Developed by Iluvatar CoreX for AI training, inference and general-purpose computing
  - Available as an air-cooled PCIe accelerator card
  - Designed for deployment in multi-card and multi-node computing systems



1.8b. Iluvatar BI-V150: A General-Purpose GPU for AI Computing

- 2. Hardware Architecture

Specification	BI-V150
GPU architecture	ivcore11
Process technology	7 nm
Device memory	64 GB HBM2e
Host interface	PCIe 4.0 ×16
Peak board power	350 W



1.8c. Iluvatar BI-V150: A General-Purpose GPU for AI Computing

- 3. Execution and Programming Model
- CUDA-Compatible, but Architecturally Different

PyTorch / Triton

↓

CoreX-adapted framework and compiler packages

↓

IXUCA runtime and vendor libraries

↓

Driver

↓

BI-V150



- Grid → Block → Warp (**64 threads**) → Thread

2. Heterogeneous Programming Fundamentals

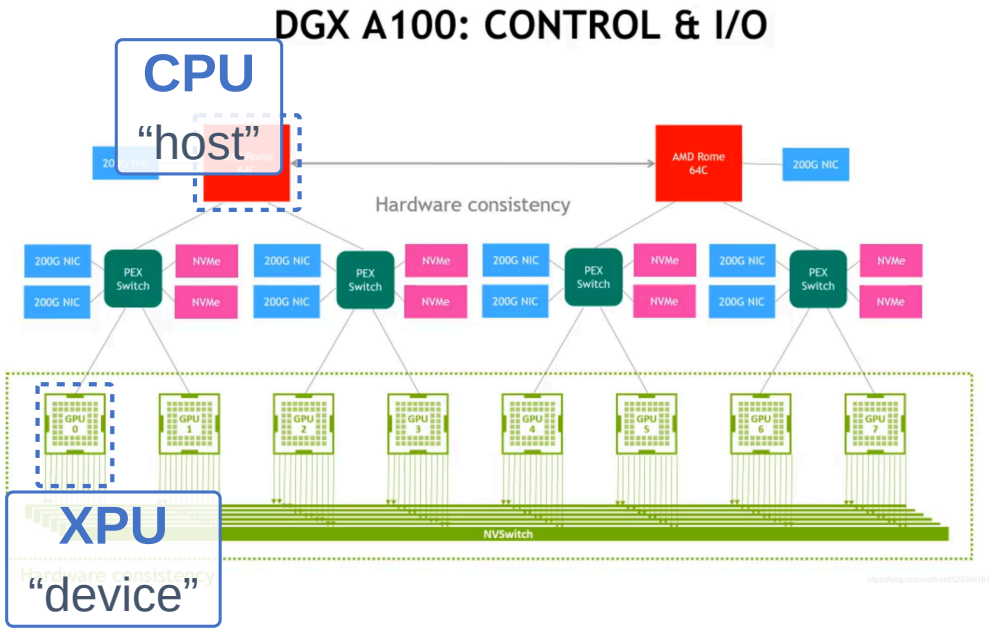
- Questions on Heterogeneous Programming
- CPU–XPU Integration and PCIe
- GPU Architecture Snapshot (Blackwell)
- Ascend DaVinci Architecture (910 $\rightarrow$ 950)
- Hosts, Streams, and Data Movement
- Expressing Parallelism and Thread Work
- GPU Memory Hierarchy
- Three Ways to Express a Kernel

2.1. Questions on Heterogeneous Programming

- How to compile the “kernel” code?
- How do CPU and GPU interact?
- How to express parallelism?
- How to express synchronization?
- How to manage and access data?
- How is kernel code executed on GPU?

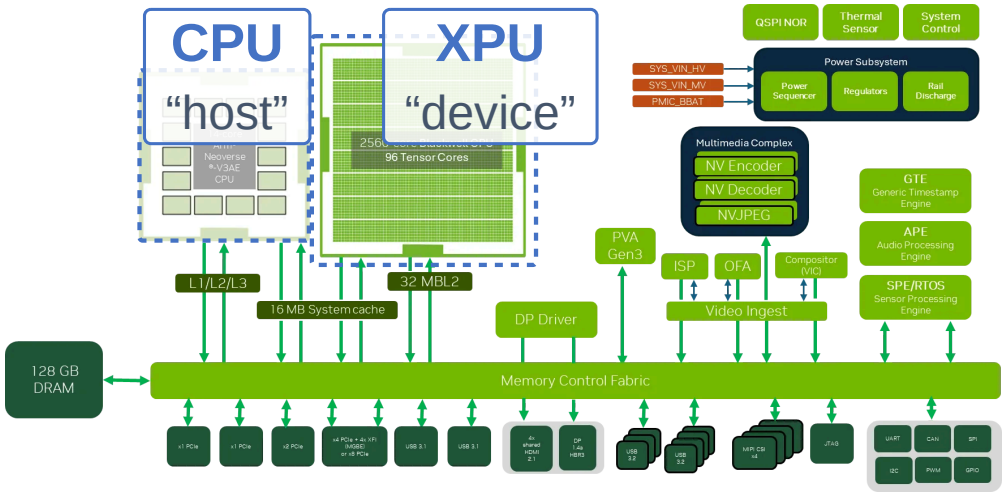
2.2. Heterogeneous Integration of CPU and XPUs

- Inter-chip (board-level) integration



- protocols: PCIe, NVLink, etc.

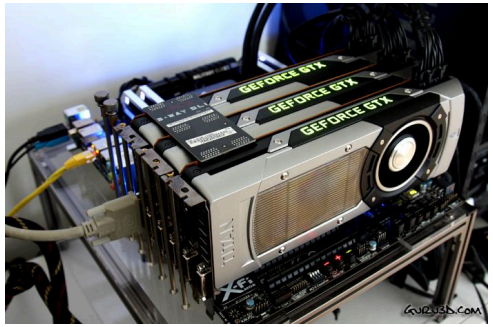
- Intra-chip integration



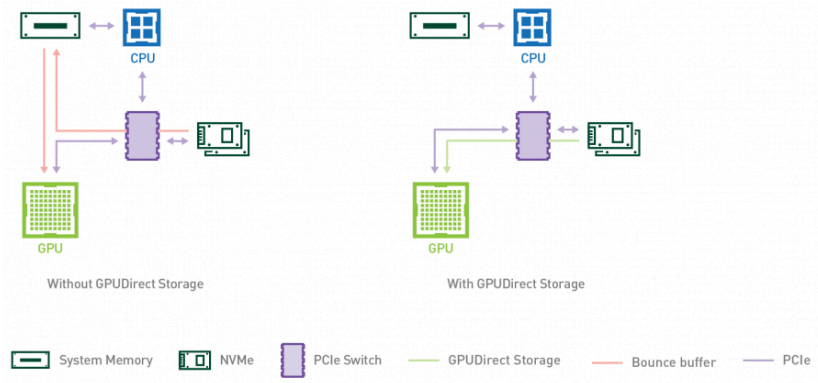
- protocols: one- or two-way coherency

2.3. PCIe Role in AI: GPU-Host Connection

- GPU connect to most Host with PCIe
  - Host→GPU: Commands (CUDA kernels, OpenCL bin)
    - Training data, Model weights…
  - GPU→Host: GPU status (kernel completion, CUDA events)
    - Computation results (gradients, inference output)…
- Traditional Data Transfer
  - 1st Copy: NVMe SSD → host Memory [with DMA]
  - 2nd Copy: host Memory → GPU Memory
- Nvidia GPUDirect Storage (from 2020.06)
  - Only 1 Copy: NVMe SSD → GPU Memory



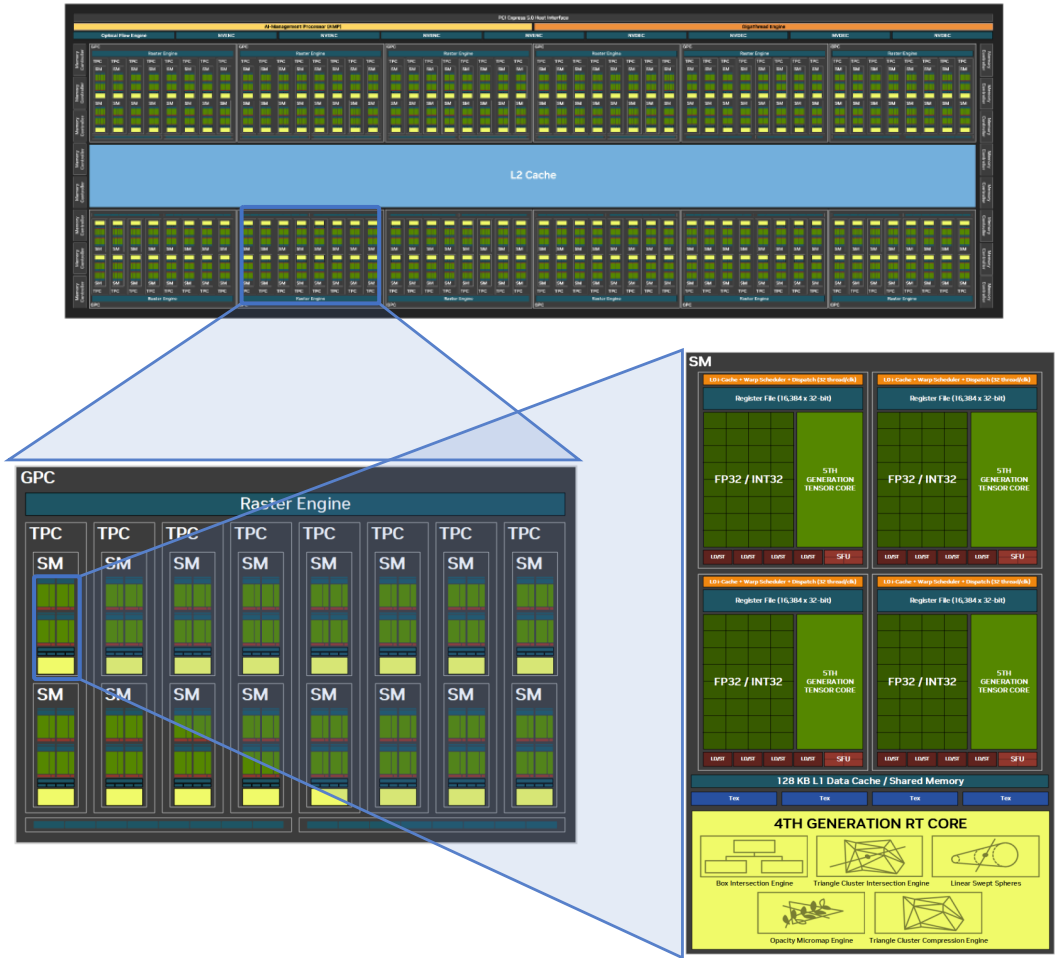
GPUs to mainboard with PCIe



2.4. GPU Architecture Snapshot (Blackwell)

Count cores (SM → GPC → chip):

- **Streaming Multiprocessor (SM)**
  - 4×32 FP32/INT32 → **128** CUDA cores
  - **4** Tensor Cores (5th-gen); 256/128 KB RF/SMEM
- **Graphics Processing Cluster (GPC)**
  - 8 TPC × 2 SM = **16** SM
  - **2048** CUDA · **64** TC ($16 \times$ SM)
- **Full GB202**
  - 12 GPC · 96 TPC → **192** SM
  - **24 576** CUDA · **768** TC ($192 \times$ SM)
- **RTX 5090** (cut-down example)
  - **170** SM → **21 760** CUDA · **680** TC
  - 96 MB L2 · 32 GB GDDR7



Why here

After PCIe connects host↔device, this hierarchy is the device side you launch onto — same vocabulary as grid / block / warp later.

NVIDIA RTX Blackwell GPU Architecture, Whitepaper, 2025

2.5. Ascend DaVinci Architecture (910 → 950)

AI Core = scalar (1D) + vector (2D) + cube (3D):

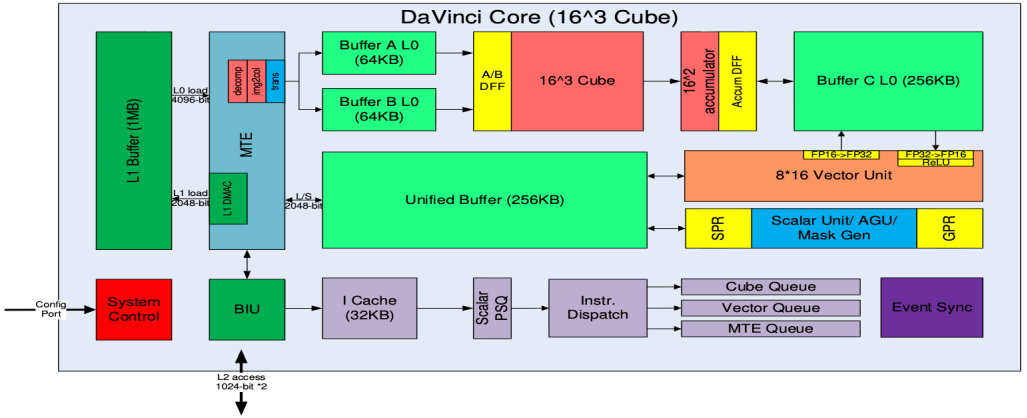
- **Cube** — systolic GEMM / conv (Tensor-Core analogue)
- **Vector / SFU** — elementwise, reduce, activations
- **Scalar + MTE** — control, address, tile DMA (UB / L0 / L1 ↔ HBM)
- **AIC vs AIV** — Cube-heavy vs Vector-heavy cores on the same mesh
- **950** — adds SIMT path on AIV; **910C** — dual-die 910B-class MCM

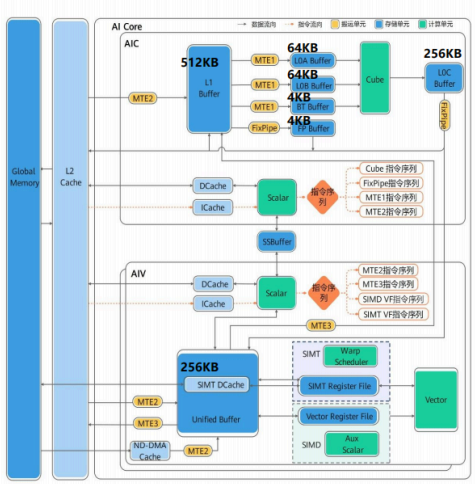
For **910 / 910B / 910C / 950**, keep one checklist:

- Cube peak & dtypes (FP16 / BF16 / INT8 / FP8-class)
- On-chip buffers (L0 / L1 / UB) and HBM generation
- Scale-up fabric / multi-die packaging when relevant

Pair with Blackwell

Same host↔device story; different engine names (Cube/MTE vs SM/TC). Software: CANN + Ascend C.





Original DaVinci AI Core (top); Ascend 950 AI Core AIC+AIV+SIMT (bottom).

2.6. Hosts Submit Work; Streams Order Kernels

- KERNEL → KERNEL
  - Kernel A writes an intermediate tensor
  - Same stream: ordered dependency
  - Kernel B reads the tensor
- Events express dependencies across streams.

- HOST → KERNEL
  - Allocate device tensors
  - Enqueue copies and kernel launches
  - Continue without waiting for completion
  - Synchronize before reading results
- A launch is usually asynchronous.

CUDA
Streams · Events · Device / Unified Memory

CANN
Device · Context · Stream · `aclrtMemcpyAsync`

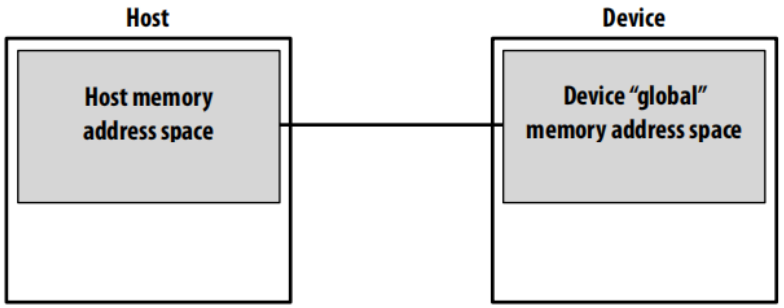
2.7. How to Manage and Access Data?

- **CUDA memory model**
  - Distinct host and device address spaces (when not using unified memory)
  - Memory allocation in GPU through `cudaMalloc()`
  - Move data between address spaces through `cudaMemcpy()`

```
int main()
{   // executed on CPU
   int A[32] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, … 31};
   int B[32] = {9, 8, 7, 6, 5, 4, 3, 2, 1, 0, … 2};
   int C[32] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, … 0};

   int *g_A, *g_B, *g_C;
   cudaMalloc((void**)&g_A, 32 * sizeof(int));
   cudaMalloc((void**)&g_B, 32 * sizeof(int));
   cudaMalloc((void**)&g_C, 32 * sizeof(int));
   cudaMemcpy(g_A, A, 32 * sizeof(int),cudaMemcpyHostToDevice);
   cudaMemcpy(g_B, B, 32 * sizeof(int),cudaMemcpyHostToDevice);

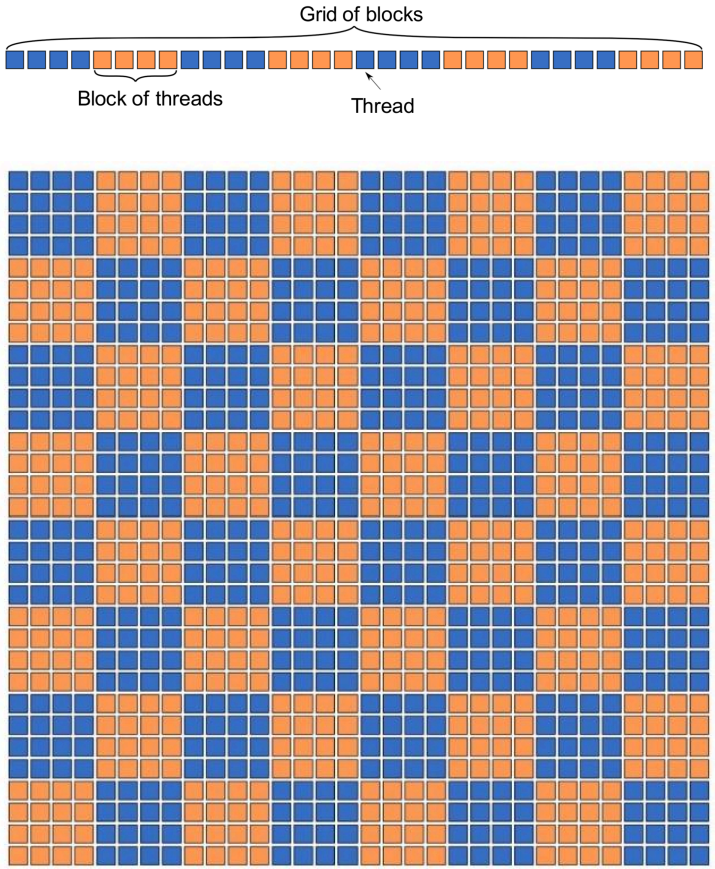
   int numBlocks = 1;
   int numThreads = 32;
   vadd <<<numBlock,numThreads>>>(g_A,g_B,g_C);
   cudaMemcpy(C, g_C, 32 * sizeof(int),cudaMemcpyDeviceToHost);
}
```



2.8. How to Express Parallelism?

- **1D Grid**
- Threads in grid have access to:
  - **Their respective block:** `blockIdx.x`
  - **Their respective thread ID in a block:** `threadIdx.x`
  - **Their block's dimension:** `blockDim.x`
  - **The number of blocks in the grid:** `gridDim.x`

```
blockIdx.x * blockDim.x + threadIdx.x
```



- **2D Grid**
- Threads in grid have access to:
  - **Their respective block IDs:** `blockIdx.x`, `blockIdx.y`
  - **Their respective thread IDs in a block:** `threadIdx.x`, `threadIdx.y`

2.9. How to Express the Work of a Thread

- A device function launched from the host is a kernel, declared with the `__global__` attribute.
- Kernels should be declared `void`.
- All pointers passed to kernels must point to memory on the device.
- Threads are grouped into warps and scheduled concurrently when resources permit.
- Each thread uses its unique thread and block IDs to compute a global ID.
- There could be more than `N` threads in the grid.
- Kernels are launched from the host:

- A simple embarrassingly parallel loop:

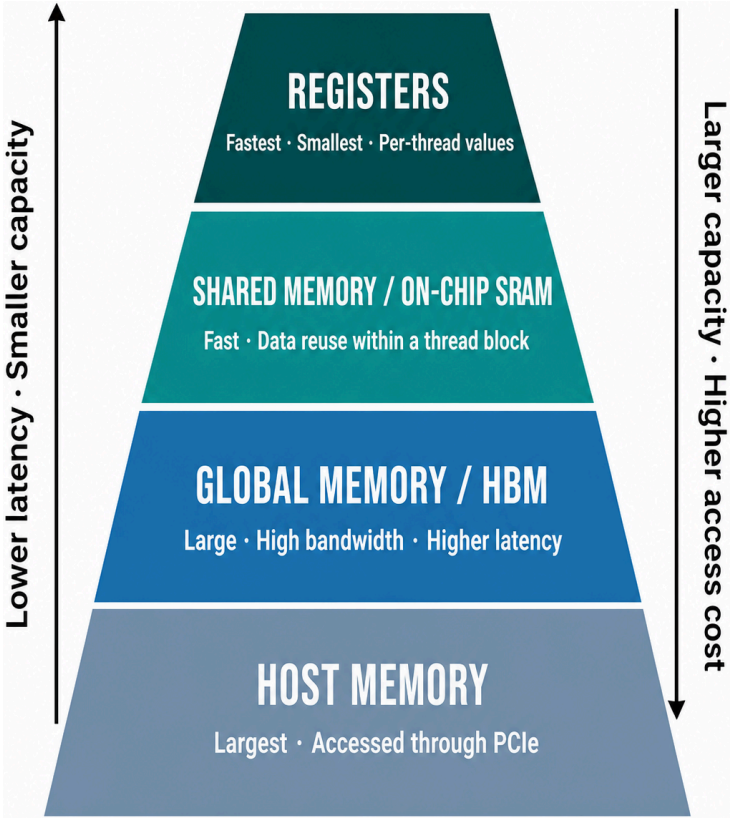
```
for (int i = 0; i < N; i++)
  h_a[i] *= 2.0;
```

- Can be translated into a GPU kernel:

```
__global__ void myKernel(int N, double *d_a)
{
  int i = threadIdx.x + blockIdx.x *
blockDim.x;
  if (i < N) d_a[i] *= 2.0;
}
```

2.10. GPU Memory Hierarchy: Capacity, Latency, and Data Reuse

- Why Does the Hierarchy Matter?
  - Global memory provides capacity, but unnecessary traffic is expensive.
  - Registers hold thread-local intermediate values.
  - Shared memory enables threads in a block to reuse data.
  - Coalesced accesses improve effective global-memory bandwidth.
  - Tiling loads data once and reuses it for multiple computations.



Lab connection: GEMM tiles reuse blocks of A and B across many multiply-accumulate operations. Larger tiles may increase reuse, but also consume more registers and on-chip resources.

2.11. Three Ways to Express a Kernel

CUDA SIMT	CUDA Tile	Ascend C
Unit: individual thread	Unit: compile-time tile	Unit: operator tile
Grid, block, `threadIdx`	Grid of tile blocks; compiler maps threads	Host tiling + AI Core blocks
Explicit indexing, shared memory and synchronization	Load, compute and store whole tiles	CopyIn $\rightarrow$ Compute $\rightarrow$ CopyOut
NVIDIA CUDA target	cuTile Python / CUDA Tile C++	Ascend AI Core / CANN

Triton is also tile-oriented, but it uses its own DSL, compiler, and target backend.

3. Triton Programming Basics

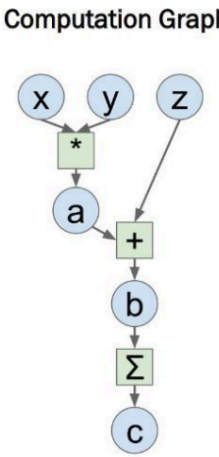
- PyTorch as the High-Level Operator Interface
- Why Triton?
- Vector Addition Case Study
- Tiles, Execution Model, and Pointer Arithmetic
- Load / Store / Masking and Reductions

3.1. PyTorch Provides the High-Level Operator Interface

- **PyTorch:** The (most) popular open-source ML library.
  - Tensor definition; GPU-enabled computation; Automatic Differentiation; Compiler Support.



Other ML Frameworks



Numpy

```
import numpy as np
np.random.seed(0)

N, D = 3, 4

x = np.random.randn(N, D)
y = np.random.randn(N, D)
z = np.random.randn(N, D)

a = x * y
b = a + z
c = np.sum(b)

grad_c = 1.0
grad_b = grad_c * np.ones((N, D))
grad_a = grad_b.copy()
grad_z = grad_b.copy()
grad_x = grad_a * y
grad_y = grad_a * x
```

Tensorflow

```
import numpy as np
np.random.seed(0)
import tensorflow as tf

N, D = 3, 4

with tf.device('/gpu:0'):
    x = tf.placeholder(tf.float32)
    y = tf.placeholder(tf.float32)
    z = tf.placeholder(tf.float32)

    a = x * y
    b = a + z
    c = tf.reduce_sum(b)

grad_x, grad_y, grad_z = tf.gradients(c, [x, y, z])

with tf.Session() as sess:
    values = {
        x: np.random.randn(N, D),
        y: np.random.randn(N, D),
        z: np.random.randn(N, D),
    }
    out = sess.run([c, grad_x, grad_y, grad_z],
                   feed_dict=values)
    c_val, grad_x_val, grad_y_val, grad_z_val = out
```

PyTorch

```
import torch

N, D = 3, 4

x = torch.rand((N, D),requires_grad=True)
y = torch.rand((N, D),requires_grad=True)
z = torch.rand((N, D),requires_grad=True)

a =x * y
b =a + z
c=torch.sum(b)

c.backward()
```

Numpy vs TensorFlow vs PyTorch

3.2. Why Triton?

- High-Performance GPU Kernels with Pythonic Ease

- Productivity-Performance Dilemma
  - Frameworks like PyTorch
    - High productivity, but “fusing” custom operators often requires using CUDA C++ to avoid overhead
  - CUDA C++
    - Offers maximum hardware utilization but is notoriously difficult to write, debug, and maintain
  - The Gap
    - Most researchers cannot afford weeks of development time to optimize a single CUDA kernel

- Triton’s “Secret Sauce”
  - Block-Level Abstraction
    - Don’t manage individual threads; you manage blocks of data.
  - Automatic Parallelization
    - The compiler handles memory coalescing, shared memory allocation, and hardware scheduling.
  - OpenAI Backing
    - Developed specifically to make GPU programming accessible for the next generation of AI models.

3.3. Case Study: Vector Addition Kernel

```python
import triton
import triton.language as tl

@triton.jit
def add_kernel(
    x_ptr, y_ptr, out_ptr, n_elements,
    BLOCK_SIZE: tl.constexpr,
):
    # 1. Which tile this program handles
    pid = tl.program_id(axis=0)
    # 2. Memory offsets for the tile
    block_start = pid * BLOCK_SIZE
    offsets = block_start + tl.arange(
        0, BLOCK_SIZE)
    # 3. Mask out-of-bounds lanes
    mask = offsets < n_elements
    # 4. Load from DRAM to registers
    x = tl.load(x_ptr + offsets, mask=mask)
    y = tl.load(y_ptr + offsets, mask=mask)
    # 5. Compute and store back to DRAM
    output = x + y
    tl.store(
        out_ptr + offsets, output, mask=mask)


BLOCK_SIZE = 256
output = torch.empty_like(x)
grid = (triton.cdiv(n_elements, BLOCK_SIZE),)

# host wrapper
add_kernel[grid](
    x, y, output, n_elements,
    BLOCK_SIZE=BLOCK_SIZE,
)
```

3.4. Triton Programs Operate on Data Tiles

- **Host side**
  - `grid` defines the number and shape of program instances
  - `kernel[grid](...)` submits work asynchronously
- **Inside each Triton program**
  - `tl.program_id` selects the problem tile
  - `tl.arange` creates vector or tile offsets
  - `tl.load` / `tl.store` move masked blocks of data
  - `tl.max`, `tl.sum` and `tl.dot` express collective tile operations
- **The compiler handles**
  - mapping tile operations to target execution lanes
  - much of the low-level coordination and instruction selection
  - You still control grid shape, masks, layout, precision and meta-parameters.
- **What Triton Simplifies**
  - Maps block/tile operations onto hardware execution lanes.
  - Handles vectorization, instruction selection, shared-memory placement, and synchronization.
- **What Remains on the Host Side**
  - PyTorch/runtime owns tensor allocation and memory placement.
  - `kernel[grid](...)` is enqueued asynchronously on the current stream.
  - Kernels exchange data through device tensors.
  - Stream ordering and events express inter-kernel dependencies.

3.5. Triton Execution Model

- From Threads to Blocks
  - CUDA Model: Focuses on Threads and Warps. Developers must manually manage synchronization and shared memory.
  - Triton Model: Focuses on Programs and Blocks. You write code that operates on a “tile” (Block) of data as a single unit.
- Key Components
  - The Grid (Coordinate Space): The Grid is a 1D, 2D, or 3D space of execution instances. When you launch a kernel, you define the grid size (e.g., `grid=(1024, )`). Each point in this grid is called a Program.
  - The Program (Instance): A Program is an independent execution instance of your kernel. Programs run in parallel across the GPU’s Compute Units. Each program identifies itself using `tl.program_id(axis)`.
  - The Block (Data Unit): A Block is a contiguous chunk of data (a “tile”) processed by one Program. Constraint: Block sizes must be powers of two (e.g., 128, 256, 512, 1024) to allow for hardware-level optimizations.

3.6. Kernel Anatomy & Pointer Arithmetic

- Defining a Triton Kernel
  - The `@triton.jit` Decorator: Transforms a Python function into a GPU-executable kernel.
  - Function Signature:
    - Pointers: Memory addresses for inputs/outputs (e.g., `x_ptr`).
    - Metadata: Size constants or strides (e.g., `n_elements`).
    - `tl.constexpr`: Compile-time arguments (e.g., `BLOCK_SIZE`).
- The Core Logic: Pointer Arithmetic
  - Identify the Program: `pid = tl.program_id(axis=0)`
  - Calculate the Start Offset: `block_start = pid * BLOCK_SIZE`
  - Generate a Range of Offsets: `offsets = block_start + tl.arange(0, BLOCK_SIZE)`
  - Construct Pointers: `pointers = base_ptr + offsets`
- Visualizing the Address Map
  - `tl.arange(0, 4)` → `[0, 1, 2, 3]`
  - `base_ptr + [0, 1, 2, 3]` → a vector of 4 memory addresses.

3.7. Memory Operations — Load, Store, and Masking

- The Data Bridge: Global Memory (DRAM) $\leftrightarrow$ SRAM/Registers
  - `tl.load(pointers, mask)`
    - Reads a block of data from the addresses specified by the pointers.
    - Efficiency: Triton automatically ensures "Coalesced Memory Access," meaning the hardware reads data in the most efficient bursts possible.
  - `tl.store(pointers, data, mask)`
    - Writes the computed results from the registers back to Global Memory.
- The "Masking" Essential
  - Because Triton requires Block Sizes to be powers of two (e.g., 512), but your actual data might not be (e.g., 500 elements), you must prevent out-of-bounds memory access.
  - The Logic: Create a boolean mask where `offsets < total_elements`.
  - The Syntax:

```
mask = offsets < n_elements
data = tl.load(ptr + offsets, mask=mask)
```

  - Benefit: If mask is False, operation is safely ignored, preventing segmentation faults.

3.8. Common Built-in Functions & Reduction

- Element-wise Math
  - Arithmetic: `+`, `-`, `*`, `/`, `**`
  - Math: `tl.exp()`, `tl.log()`, `tl.sqrt()`, `tl.sin()`, `tl.cos()`
  - Comparison: `==`, `!=`, `<`, `>`, `tl.where(condition, x, y)`
- Reduction Operations
  - `tl.sum(data, axis=...)`: Computes the sum along the specified axis.
  - `tl.max(data, axis=...)`: Finds the maximum value.
  - `tl.argmin()` / `tl.argmax()`: Finds the index of the min/max.
- Atomic Operations
  - `tl.atomic_add(pointer, value, mask)`
  - `tl.atomic_max(pointer, value, mask)`

4. Lab: Softmax, GEMM, and Tuning

- Softmax equation · GEMM pseudocode
- Softmax: one program per row
- GEMM tile reuse across the K-loop
- Debug & correctness
- Controlled GEMM tuning experiments

4.1. Softmax Equation · GEMM Pseudocode

Softmax (stable, per row i)

$$\mathrm{softmax}(X)_{ij} = \exp\big(X_{ij} - m_i\big) / \sum_k \exp(X_{ik} - m_i)$$

where $m_i = \max_k X_{ik}$

- Rows independent $\rightarrow$ one program per row
- Subtract row max before `exp`
- Prefer fp32 for max / sum (even if I/O is fp16)

Blocked GEMM ($C = AB$)

```
# Parallel over output tiles (M × N):
for m in range(0, M, BLOCK_M):
  for n in range(0, N, BLOCK_N):
    acc = zeros((BM, BN), dtype=fp32)
    for k in range(0, K, BLOCK_K):
      a = A[m:m+BM, k:k+BK]
      b = B[k:k+BK, n:n+BN]
      acc += dot(a, b)
    C[m:m+BM, n:n+BN] = acc
```

Outer $m, n \rightarrow$ Triton grid; inner $K \rightarrow$ loop in the kernel.

4.2. Softmax: One Program per Row

```python
@triton.jit
def softmax_kernel(
    out_ptr, x_ptr, row_stride, n_cols,
    BLOCK_SIZE: tl.constexpr,
):
    # One program owns one row
    row = tl.program_id(0)
    cols = tl.arange(0, BLOCK_SIZE)
    mask = cols < n_cols
    x = tl.load(
        x_ptr + row * row_stride + cols,
        mask=mask, other=-float("inf"),
    )
```

```python
# Safe Softmax: max -> exp -> sum -> normalize
x = x - tl.max(x, axis=0)
num = tl.exp(x)
den = tl.sum(num, axis=0)
tl.store(
    out_ptr + row * row_stride + cols,
    num / den, mask=mask,
)

# Host: one program instance per row
softmax_kernel[(n_rows,)](
    out, x, x.stride(0), n_cols,
    BLOCK_SIZE=triton.next_power_of_2(n_cols),
)
```

Pad invalid lanes with `-inf` for max.

4.3. GEMM Reuses Tiles Across the K-loop

```
# One program computes one C[BM, BN] tile
pid_m = ...
pid_n = ...
offs_m = pid_m * BM + tl.arange(0, BM)
offs_n = pid_n * BN + tl.arange(0, BN)
offs_k = tl.arange(0, BK)

acc = tl.zeros((BM, BN), dtype=tl.float32)

for k in range(0, tl.cdiv(K, BK)):
    a = tl.load(
        a_ptrs,
        mask=(offs_m[:, None] < M)
             & (k * BK + offs_k[None, :] < K),
        other=0.0,
    )
    b = tl.load(
        b_ptrs,
        mask=(k * BK + offs_k[:, None] < K)
             & (offs_n[None, :] < N),
        other=0.0,
    )
    acc += tl.dot(a, b)
    a_ptrs += BK * stride_ak
    b_ptrs += BK * stride_bk

tl.store(
    c_ptrs, acc.to(tl.float16), mask=c_mask)

# Three boundaries matter: M, N and K.
```

4.4. Debug & Correctness

Compile / device

- `tl.static_assert` · `tl.static_print`
- `tl.device_assert` · `tl.device_print`
- Device asserts need `TRITON_DEBUG`

Interpreter

```
TRITON_INTERPRET=1 python test.py
```

- Sequential NumPy semantics
- Ordinary `print` / `pdb`
- Logic bugs before the GPU

Check with PyTorch

- `torch.allclose(..., atol, rtol)`
- `torch.autograd.gradcheck(...)`
- Exact blocks **and** awkward edges (`N=1`, `N=BLOCK±1`)

```python
class MyOp(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, y):
        z = my_kernel(x, y)
        ctx.save_for_backward(x, y)
        return z
    @staticmethod
    def backward(ctx, grad_z):
        x, y = ctx.saved_tensors
        ...  # separate Triton backward
```

Order

- static → interpret → device,
- then `allclose` / `gradcheck`.
- Never time a cold launch.

4.5. Tune GEMM with Controlled Experiments

```python
# PASS tiles only (from block sweep)
AUTOTUNE_CONFIGS = [
  triton.Config(
    {"BLOCK_M": 64, "BLOCK_N": 64, "BLOCK_K": 32},
    num_warps=4, num_stages=3,
  ),
  triton.Config(
    {"BLOCK_M": 128, "BLOCK_N": 64, "BLOCK_K": 32},
    num_warps=8, num_stages=3,
  ),
  # ...
]

@triton.autotune(
  configs=AUTOTUNE_CONFIGS,
  key=["M", "N", "K"],
)
@triton.jit
def matmul_autotuned_kernel(
    a_ptr, b_ptr, c_ptr, M, N, K, ...,
    BLOCK_M: tl.constexpr,
    BLOCK_N: tl.constexpr,
    BLOCK_K: tl.constexpr,
):
    ...
```

```python
def matmul_autotuned(a, b, out=None):
    M, K = a.shape; N = b.shape[1]
    c = out or torch.empty((M, N), ...)
    grid = lambda meta: (
        triton.cdiv(M, meta["BLOCK_M"]),
        triton.cdiv(N, meta["BLOCK_N"]),
    )
    matmul_autotuned_kernel[grid](
        a, b, c, M, N, K, ...)
    return c

# Confirm winner vs exact-config fixed launch
tile = (... from .best_config ...)
matmul_fixed(a, b, *tile, ...)
```

Discipline

baseline (correct → warmup → sync → median) → sweep → **PASS-only** autotune → confirm winner. PASS ≠ fastest everywhere.

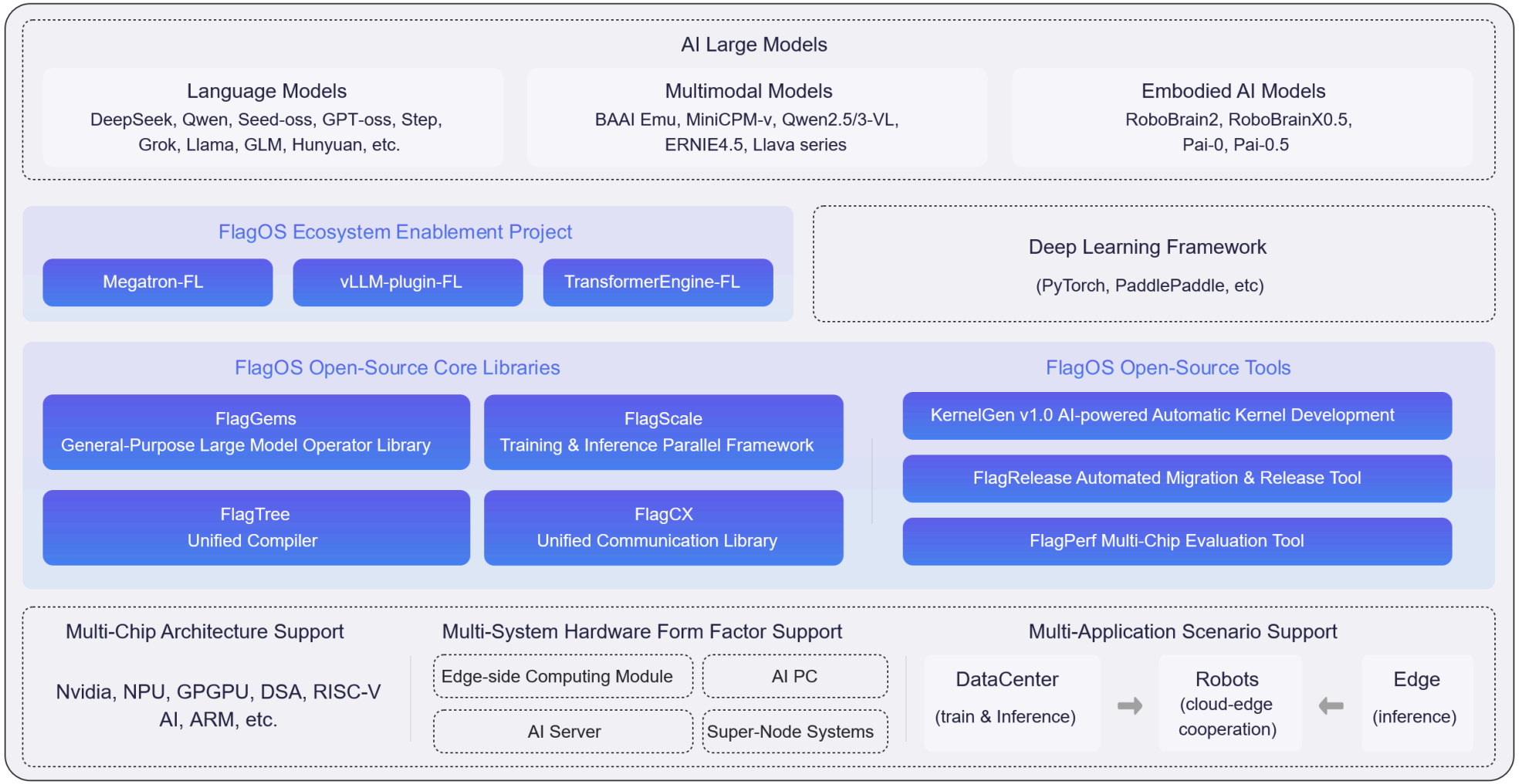
Domestic AI Accelerators (FlagTree Ecosystem)

- Domestic NPU / GPU vendors with backends in **FlagTree** (unified Triton multi-backend compiler).
- Same GEMM / memory principles; different ISA, memory hierarchy, and software stacks.

Huawei Ascend 华为昇腾	Cambricon 寒武纪	Enflame 燧原	ILUVATAR 天数智芯
HYGON 海光信息	Moore Threads 摩尔线程	MetaX 沐曦	T-Head 平头哥
KLX / Kunlunxin 昆仑芯	SpacemiT 进迭时空	Tsingmicro 清微智能	Huixi 辉羲智能
Sunrise 曦望芯科	DAMO Thrive 达摩院	ARM China 安谋科技	Biren† 壁仞

flagos-ai/flagtree · Logos are trademarks of their owners (teaching use only).

The Solution: FlagOS, a Unified AI Software Stack



African Scientific Research and Innovation Council

BAAI 智源研究院

北京大学 PEKING UNIVERSITY

中国人民大学 RENMIN UNIVERSITY OF CHINA

Thanks

Questions & further discussion welcome

Guojie Luo
罗国杰
gluo@pku.edu.cn

Jing Wang
王晶
jwang@ruc.edu.cn

BAAI Team
智源团队
github.com/flagos-ai



Supported by UNDP.